

Hospital Patient Management

Using Linked Lists

Introduction

In modern hospital management, efficient handling of patient information is crucial, especially when prioritizing critical patients over normal ones. A linked list is an appropriate data structure for this scenario due to its dynamic size and flexibility, which allow insertion, deletion and rearrangement of patient nodes without significant overhead. By modeling each patient as a node in a linked list, it becomes easier to implement operations such as sorting based on priority, traversing all patients and performing dynamic updates, reflecting real-world constraints in a programmatic solution

Task 1. Creation of the Linked List

i. Hand Method:

Each patient is represented by a node containing the Patient ID and a pointer to the next node. Initially, patients are linked sequentially from floor 1 to 20:

Head → 1 → 2 → 3 → 4 → 5 → ... → 16 → 17 → 18 → 19 → 20 → NULL

ii. Code Implementation:

```
C++ 1.cpp > ...
1  #include <iostream>
2  using namespace std;
3
4  struct Node {
5      int patientID;
6      Node* next;
7  };
8
9  Node* createLinkedList() {
10     Node* head = nullptr;
11     Node* tail = nullptr;
12     for (int i = 1; i <= 20; i++) {
13         Node* newNode = new Node{ i, nullptr };
14         if (!head) head = tail = newNode;
15         else { tail->next = newNode; tail = newNode; }
16     }
17     return head;
18 }
19
20 void printList(Node* head) {
21     while (head) { cout << head->patientID << " "; head = head->next; }
22     cout << endl;
23 }
24
25 int main() {
26     Node* head = createLinkedList();
27     cout << "Initial Linked List: ";
28     printList(head);
29     return 0;
30 }
```

Task 2. Rearranging Odd (Critical) Before Even (Normal)

i. Hand Method:

Separated odd-numbered (critical) patients and even-numbered (normal) patients into two lists. Connected the odd list to the even list to prioritize critical patients

Odd List: $1 \rightarrow 3 \rightarrow 5 \rightarrow \dots \rightarrow 19$

Even List: $2 \rightarrow 4 \rightarrow 6 \rightarrow \dots \rightarrow 20$

Final List: $1 \rightarrow 3 \rightarrow 5 \rightarrow \dots \rightarrow 19 \rightarrow 2 \rightarrow 4 \rightarrow \dots \rightarrow 20 \rightarrow \text{NULL}$

ii. Code Implementation:

```
C++ 2.cpp > ...
1  #include <iostream>
2  using namespace std;
3
4  struct Node {
5      int patientID;
6      Node* next;
7  };
8
9  Node* createlinkedList() {
10     Node* head = nullptr;
11     Node* tail = nullptr;
12     for (int i = 1; i <= 20; i++) {
13         Node* newNode = new Node{ i, nullptr };
14         if (!head) head = tail = newNode;
15         else { tail->next = newNode; tail = newNode; }
16     }
17     return head;
18 }
19
20 Node* rearrangeOddEven(Node* head) {
21     Node* oddHead = nullptr, *oddTail = nullptr;
22     Node* evenHead = nullptr, *evenTail = nullptr;
23     Node* current = head;
24     while (current) {
25         if (current->patientID % 2 != 0) {
26             if (!oddHead) oddHead = oddTail = current;
27             else { oddTail->next = current; oddTail = current; }
28         } else {
29             if (!evenHead) evenHead = evenTail = current;
30             else { evenTail->next = current; evenTail = current; }
31         }
32         current = current->next;
33     }
34     if (oddTail) oddTail->next = evenHead;
35     if (evenTail) evenTail->next = nullptr;
36     return oddHead ? oddHead : evenHead;
37 }
38
39 void printList(Node* head) {
40     while (head) { cout << head->patientID << " "; head = head->next; }
41     cout << endl;
42 }
43
44 int main() {
45     Node* head = createlinkedList();
46     head = rearrangeOddEven(head);
47     cout << "After rearranging odd first: ";
48     printList(head);
49     return 0;
50 }
```

Task 3. Sorting the Linked List (Ascending Order)

i. Hand Method:

Compared each node with all subsequent nodes and swapped values when required...

Sorted list: $1 \rightarrow 2 \rightarrow 3 \rightarrow \dots \rightarrow 20 \rightarrow \text{NULL}$

ii. Code Implementation:

```

C++ 3.cpp > ...
1  #include <iostream>
2  using namespace std;
3
4  struct Node {
5      int patientID;
6      Node* next;
7  };
8
9  Node* createLinkedList() {
10     Node* head = nullptr;
11     Node* tail = nullptr;
12     for (int i = 1; i <= 20; i++) {
13         Node* newNode = new Node{ i, nullptr };
14         if (!head) head = tail = newNode;
15         else { tail->next = newNode; tail = newNode; }
16     }
17     return head;
18 }
19
20 Node* sortLinkedList(Node* head) {
21     if (!head) return nullptr;
22     bool swapped;
23     do {
24         swapped = false;
25         Node* current = head;
26         while (current->next) {
27             if (current->patientID > current->next->patientID) {
28                 swap(current->patientID, current->next->patientID);
29                 swapped = true;
30             }
31             current = current->next;
32         }
33     } while (swapped);
34     return head;
35 }
36
37 void printList(Node* head) {
38     while (head) { cout << head->patientID << " "; head = head->next; }
39     cout << endl;
40 }
41
42 int main() {
43     Node* head = createLinkedList();
44     head = sortLinkedList(head);
45     cout << "Sorted Linked List: ";
46     printList(head);
47     return 0;
48 }

```

Task 4. Sum of Patient IDs

i. Hand Method:

Sum = $1 + 2 + 3 + \dots + 20 = (20 * 21) / 2 = 210$

ii. Code Implementation:

```

C++ 4.cpp > ...
1  #include <iostream>
2  using namespace std;
3
4  struct Node {
5      int patientID;
6      Node* next;
7  };
8
9  Node* createLinkedList() {
10     Node* head = nullptr;
11     Node* tail = nullptr;
12     for (int i = 1; i <= 20; i++) {
13         Node* newNode = new Node{ i, nullptr };
14         if (!head) head = tail = newNode;
15         else { tail->next = newNode; tail = newNode; }
16     }
17     return head;
18 }
19
20 int sumLinkedList(Node* head) {
21     int sum = 0;
22     while (head) { sum += head->patientID; head = head->next; }
23     return sum;
24 }
25
26 int main() {
27     Node* head = createLinkedList();
28     cout << "Sum of patient IDs: " << sumLinkedList(head) << endl;
29     return 0;
30 }

```

Task 5. Identifying the Middle (or nearest middle) Patient

i. Hand Method:

With 20 nodes, middle patient is either 10th or 11th node → Patient ID = 10 or 11

ii. Code Implementation:

```
C++ 5.cpp > ...
1  #include <iostream>
2  using namespace std;
3
4  struct Node {
5      int patientID;
6      Node* next;
7  };
8
9  Node* createLinkedList() {
10     Node* head = nullptr;
11     Node* tail = nullptr;
12     for (int i = 1; i <= 20; i++) {
13         Node* newNode = new Node( i, nullptr );
14         if (!head) head = tail = newNode;
15         else { tail->next = newNode; tail = newNode; }
16     }
17     return head;
18 }
19
20 Node* findMiddle(Node* head) {
21     Node* slow = head;
22     Node* fast = head;
23     while (fast && fast->next) {
24         slow = slow->next;
25         fast = fast->next->next;
26     }
27     return slow;
28 }
29
30 int main() {
31     Node* head = createLinkedList();
32     Node* middle = findMiddle(head);
33     if (middle) cout << "Middle Patient ID: " << middle->patientID << endl;
34     return 0;
35 }
```

Task 6. Removing All Normal (Even) Patients

i. Hand Method:

Traversed and removed nodes with even IDs and then reconnected odd nodes...

Remaining list: 1 → 3 → 5 → 7 → 9 → 11 → 13 → 15 → 17 → 19 → NULL

ii. Code Implementation:

```
C++ 6.cpp > ...
1  #include <iostream>
2  using namespace std;
3
4  struct Node {
5      int patientID;
6      Node* next;
7  };
8
9  Node* createLinkedList() {
10     Node* head = nullptr;
11     Node* tail = nullptr;
12     for (int i = 1; i <= 20; i++) {
13         Node* newNode = new Node( i, nullptr );
14         if (!head) head = tail = newNode;
15         else { tail->next = newNode; tail = newNode; }
16     }
17     return head;
18 }
19
20 Node* removeEvenPatients(Node* head) {
21     while (head && head->patientID % 2 == 0) head = head->next;
22     Node* current = head;
23     while (current && current->next) {
24         if (current->next->patientID % 2 == 0)
25             current->next = current->next->next;
26         else
27             current = current->next;
28     }
29     return head;
30 }
31
32 void printList(Node* head) {
33     while (head) { cout << head->patientID << " "; head = head->next; }
34     cout << endl;
35 }
36
37 int main() {
38     Node* head = createLinkedList();
39     head = removeEvenPatients(head);
40     cout << "Linked List after removing even patients: ";
41     printList(head);
42     return 0;
43 }
```

Task 7. Inserting a New Highest-Priority Critical Patient

i. Hand Method:

Inserted new critical patient (patient ID = 0) at the head

$0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow \dots \rightarrow 20 \rightarrow \text{NULL}$

ii. Code Implementation:

```
C++ 7.cpp > ...
1  #include <iostream>
2  using namespace std;
3
4  struct Node {
5      int patientID;
6      Node* next;
7  };
8
9  Node* createLinkedList() {
10     Node* head = nullptr;
11     Node* tail = nullptr;
12     for (int i = 1; i <= 20; i++) {
13         Node* newNode = new Node( i, nullptr );
14         if (!head) head = tail = newNode;
15         else { tail->next = newNode; tail = newNode; }
16     }
17     return head;
18 }
19
20 Node* insertCriticalPatient(Node* head, int newID) {
21     Node* newNode = new Node( newID, head );
22     return newNode;
23 }
24
25 void printList(Node* head) {
26     while (head) { cout << head->patientID << " "; head = head->next; }
27     cout << endl;
28 }
29
30 int main() {
31     Node* head = createLinkedList();
32     head = insertCriticalPatient(head, 0);
33     cout << "After inserting new critical patient: ";
34     printList(head);
35     return 0;
36 }
```

Task 8. Searching for Critical Patient with a Normal Adjacent

i. Hand Method:

Traverse and identify an odd patient followed by an even patient...

Example: Patient 1 (odd) is adjacent to 2 (even)

ii. Code Implementation:

```
C++ 8.cpp > ...
1  #include <iostream>
2  using namespace std;
3
4  struct Node {
5      int patientID;
6      Node* next;
7  };
8
9  Node* createLinkedList() {
10     Node* head = nullptr;
11     Node* tail = nullptr;
12     for (int i = 1; i <= 20; i++) {
13         Node* newNode = new Node( i, nullptr );
14         if (!head) head = tail = newNode;
15         else { tail->next = newNode; tail = newNode; }
16     }
17     return head;
18 }
19
20 Node* searchCriticalAdjacent(Node* head) {
21     Node* current = head;
22     while (current && current->next) {
23         if (current->patientID % 2 != 0 && current->next->patientID % 2 == 0)
24             return current;
25         current = current->next;
26     }
27     return nullptr;
28 }
29
30 int main() {
31     Node* head = createLinkedList();
32     Node* found = searchCriticalAdjacent(head);
33     if (found)
34         cout << "Critical patient with normal adjacent: " << found->patientID << endl;
35     else
36         cout << "No such patient found." << endl;
37     return 0;
38 }
```

Task 9. Converting to Doubly Linked List and Adding a New Head

i. Hand Method:

Each node now has prev and next pointers, inserted new head for highest-priority patient

ii. Code Implementation:

```
C++ 9.cpp > ...
1  #include <iostream>
2  using namespace std;
3
4  struct Node {
5      int patientID;
6      Node* next;
7  };
8
9  struct DNode {
10     int patientID;
11     DNode* prev;
12     DNode* next;
13 };
14
15 Node* createLinkedList() {
16     Node* head = nullptr;
17     Node* tail = nullptr;
18     for (int i = 1; i <= 20; i++) {
19         Node* newNode = new Node{ i, nullptr };
20         if (!head) head = tail = newNode;
21         else { tail->next = newNode; tail = newNode; }
22     }
23     return head;
24 }
25
26 DNode* convertToDoubly(Node* head) {
27     if (!head) return nullptr;
28     DNode* dHead = new DNode{ head->patientID, nullptr, nullptr };
29     DNode* prev = dHead;
30     Node* current = head->next;
31
32     while (current) {
33         DNode* temp = new DNode{ current->patientID, prev, nullptr };
34         prev->next = temp;
35         prev = temp;
36         current = current->next;
37     }
38     return dHead;
39 }
40
41 void printDoubly(DNode* head) {
42     while (head) {
43         cout << head->patientID << " ";
44         head = head->next;
45     }
46     cout << endl;
47 }
48
49 int main() {
50     Node* head = createLinkedList();
51     DNode* dHead = convertToDoubly(head);
52     cout << "Doubly linked list: ";
53     printDoubly(dHead);
54     return 0;
55 }
```

Conclusion

This assignment demonstrates the design, implementation and evaluation of a linked list-based hospital patient management system. By performing operations such as rearrangement, sorting, insertion, deletion and conversion to a doubly linked list, it highlights the efficiency and suitability of linked lists for dynamic and priority-based data. Overall, the task improved algorithmic thinking, data structure application and problem-solving skills, fully aligned with real-world constraints

Analytical Questions:

1. Explain why the linked list data structure is particularly suitable for this hospital patient management system.

Linked lists are highly suitable for hospital patient management because they provide flexibility, efficient memory use and dynamic operations, which are essential for handling critical and normal patients with varying priorities. Detailed reasoning is as follows:

➤ **Dynamic Memory Allocation**

Linked lists do not require pre-allocation of memory, nodes are created only when needed. As a result, when a new critical patient arrives, a new node can be inserted without shifting existing nodes, unlike arrays

➤ **Efficient Insertion and Deletion**

Adding or removing patients is straightforward. Insertion at the head is $O(1)$ and deletion at any position requires only pointer adjustments. Thus, removing all normal (even-numbered) patients or inserting a highest-priority patient at the front can be done efficiently

➤ **Easy Priority-Based Rearrangement**

Critical patients (odd-numbered) can be moved to the front while maintaining relative order, without reallocating memory. So rearranging patients for doctor visits is simple using pointer adjustments

➤ **Simplified Traversal for Daily Visits**

Doctors can traverse the linked list sequentially from head to tail to visit patients floor by floor, which mirrors real-world workflow

➤ **Real-World Applicability**

Hospitals experience dynamic patient admissions, discharges and priority changes. Linked lists offer scalable and reliable management of such dynamic data

Thus, linked lists provide a flexible, efficient and scalable solution for managing hospital patients, allowing priority handling, dynamic operations and minimal memory overhead

2. Discuss how selecting an inappropriate data structure could negatively impact system performance and resource utilization.

Selecting the wrong data structure can severely degrade system efficiency, memory usage and operational reliability. The consequences are detailed below:

(a) **Using Arrays Instead of Linked Lists**

- Fixed-size arrays may allocate unnecessary memory or risk overflow if patient count increases
- Adding or deleting patients in an array requires shifting elements $\rightarrow O(n)$ operations. For example, deleting 10 normal patients in an array requires moving all subsequent elements for each deletion, increasing errors and time

(b) **Poor Priority Handling**

- Arrays cannot easily support dynamic priority-based rearrangement

- Moving critical patients ahead of normal patients would involve multiple swaps and data shifts, which is inefficient and error-prone

(c) Impact on Performance

- High operation costs slow program execution, especially as patient count grows
- Real-time hospital systems require fast insertions, deletions and searches, which arrays fail to provide efficiently

(d) Resource Utilization

- Using inappropriate structures may overuse CPU and memory, causing the system to lag or crash
- Linked lists allow $O(1)$ insertions and deletions at critical positions, which arrays cannot achieve efficiently

Managing 1000 patients with frequent priority changes using arrays requires repeated shifting of many elements, making the approach highly inefficient. Such inefficiency shows how poor data structure selection leads to performance issues and wasted memory. On the other hand, linked lists update priorities through simple pointer adjustments, providing a faster, more reliable and memory-efficient solution for dynamic patient management